



جامعة دمشق

كلية الهندسة المعلوماتية

السنة الرابعة

قسم الذكاء الاصطناعي

مشروع مقرر البرمجة التفرعية محرك المعالجة عالي الأداء لنظام التجارة الالكترونية

High-Performance E-Commerce Backend Engine

إعداد الطلاب :

حسين علي شحادة

ضياء الدين عصام العلي

لانا محمد شريط

يمامة مناضل حمورة

20/6/2026

المقدمة

مع ازدياد الاعتماد على أنظمة التجارة الإلكترونية أصبحت القدرة على معالجة عدد كبير من الطلبات المتزامنة والمحافظة على سلامة البيانات من أهم التحديات التي تواجه الأنظمة الخلفية الحديثة. تتطلب هذه الأنظمة آليات فعالة للتحكم بالتزامن وإدارة الموارد وتحسين الأداء وضمان استقرار الخدمة تحت الضغط.

يهدف هذا المشروع إلى تطوير محرك معالجة عالي الأداء لمنصة تجارة إلكترونية اعتماداً على إطار العمل Django وقاعدة البيانات PostgreSQL ، مع تطبيق مجموعة من مفاهيم البرمجة المتوازية والمدرسة ضمن المقرر. يركز المشروع على تحقيق المتطلبات غير الوظيفية المتعلقة بحماية البيانات المشتركة، إدارة الموارد، تنفيذ المهام غير المتزامنة، التخزين المؤقت، توزيع الأحمال، سلامة المعاملات، واختبار الأداء تحت الضغط.

تم استخدام مجموعة من التقنيات الحديثة مثل Redis و Celery و Nginx و Locust لتحقيق هذه الأهداف، مع إجراء اختبارات عملية لإثبات فاعلية الحلول المطبقة وقياس أثرها على أداء النظام واستقراره.

أهداف المشروع

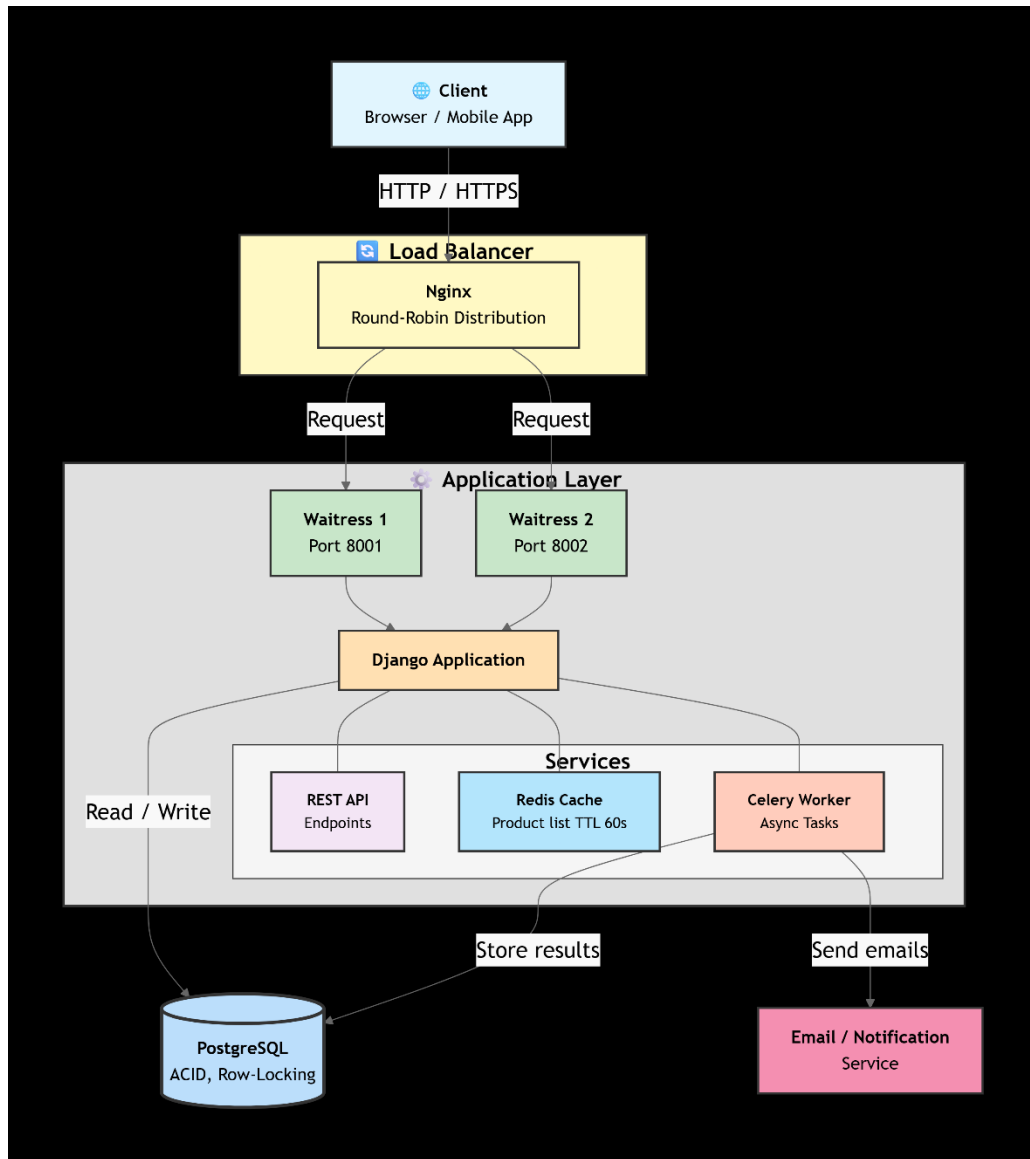
يهدف المشروع إلى تحقيق الأهداف التالية:

1. بناء واجهة خلفية (Backend) لمنصة تجارة إلكترونية تعتمد على REST API.
2. حماية البيانات المشتركة من التضارب أثناء الوصول المتزامن.
3. إدارة الموارد الحاسوبية ومنع استنزافها تحت الأحمال المرتفعة.
4. نقل العمليات طويلة التنفيذ إلى مهام تعمل في الخلفية.
5. معالجة البيانات الضخمة على شكل دفعات (Batch Processing).
6. تحسين زمن الاستجابة باستخدام التخزين المؤقت (Caching).
7. تحقيق سلامة المعاملات وفق مبادئ ACID.
8. دعم التوسع الأفقي من خلال توزيع الأحمال بين عدة خوادم.
9. اختبار استقرار النظام تحت الضغط وتحليل نقاط الاختناق.
10. تطبيق المفاهيم النظرية للبرمجة المتوازية ضمن مشروع عملي متكامل.

• وتم ذلك من خلال تطبيق مفاهيم البرمجة المتوازية التي تمت دراستها خلال المادة وهي :

- 1) Concurrency control(pessimistic and optimistic locking)
- 2) Resource management (semaphore limiting concurrent orders)
- 3) Asynchronous task offloading(celery)
- 4) Batch processing(processing in chunks "daily sales report")
- 5) Load balancing(nginx + multiple waitress workers)
- 6) Distributed caching (redis/memurai)
- 7) Transaction integrity(ACID)
- 8) Stress testing (locust)

بنية النظام (System Architecture)



يتكون النظام من مجموعة من المكونات المتكاملة التي تعمل معاً لتقديم خدمة مستقرة وعالية الأداء:

المحرك الأساسي للنظام للعمليات الأساسية مثل عرض المنتجات وتقديم rest api	django
قاعدة البيانات للنظام تدعم الاقفال على مستوى الصف ومعاملات ACID	postgreSQL
يوزع الحمل او الطلبات بطريقة البروكسي العكسي على خوادم التطبيق	Nginx
تعمل ك مخدم للتطبيق على بورتات النظام المختلفة	waitress
ذاكرة تخزين مؤقتة للكاشينغ ووسيط ل celery	Redis/memurai
يقوم بتنفيذ المهم المجدولة مثل الاشعارات او صدار التقارير	celery
أداة لاختبار النظام تحت الضغط	locust

سير عمل الطلبات

يرسل العميل طلبا الى Nginx على البورت 80

يقوم Nginx بتوجيه الطلب الى احدى ال waitress بالتناوب

يعالج Django الطلب بحسب نوع ال api :

1. Get/api/products

يغيد البيانات من قاعدة البيانات او اليانات المخزنة فس redis في حالة cache HIT

2. Post/api/order

يحصل على semaphore ثم يبدأ العمل مع قاعدة البيانات لتحديث القيم بعد تفعيل celery

3. Post/api/restock

يحدث قاعدة البيانات فقط عندما تكون ال version مطابقة optimistic locking

4. Pot/api /batch/daily_report

يضع المهمة في queue ل celery كي تتم المعالجة على دفعات

ثم يعود الرد الى العميل بينما ينفذ celery في الخلفية

• المتطلبات غير الوظيفية.

1. حماية البيانات المشتركة من التضارب

• المتطلب:

منع حدوث تضارب عند محاولة عدة مستخدمين تعديل نفس البيانات المشتركة، مثل كمية المخزون الخاصة بالمنتجات.

• طريقة التطبيق:

تم تطبيق القفل التشاؤمي (Pessimistic Locking) داخل دالة إنشاء الطلب، حيث يتم قفل سجل المنتج باستخدام `select_for_update()` قبل تعديل كمية المخزون. وبذلك لا يُسمح إلا لطلب واحد بالوصول إلى السجل وتعديله في اللحظة نفسها، بينما تنتظر الطلبات الأخرى حتى انتهاء العملية الحالية.

• التبرير:

يضمن هذا الأسلوب سلامة البيانات ومنع حدوث Race Conditions حتى في حالات الضغط العالي وعدد المستخدمين الكبير، كما يتميز بسهولة التطبيق والاعتماد على آليات القفل المدمجة في قاعدة البيانات PostgreSQL.

```
with transaction.atomic():  
    product = Product.objects.select_for_update().get(id=product_id)  
    if product.stock < quantity:  
        return JsonResponse({'error': 'Out of stock'}, status=400)  
    product.stock -= quantity
```

• نتيجة الاختبار:

تم تنفيذ خمسة طلبات متزامنة على منتج يملك قطعة واحدة فقط في المخزون. أظهرت النتائج نجاح طلب واحد فقط، بينما فشلت الطلبات الأربعة الأخرى بسبب نفاد الكمية، مما يؤكد نجاح آلية الحماية من التضارب.

```
=====
NFR #1: RACE CONDITION PROTECTION TEST
=====

X OUT OF STOCK (400): Race prevented
X OUT OF STOCK (400): Race prevented
X OUT OF STOCK (400): Race prevented
X OUT OF STOCK (400): Race prevented
✓SUCCESS (200): Order placed

[RESULTS]
Successes: 1 (expected: 1)
Out of stock: 4 (expected: 4)
Errors: 0 (expected: 0)

✓NFR #1 PASSED
```


// إدارة الموارد الحاسوبية

- المتطلب:

التحكم بعدد العمليات المتزامنة التي ينفذها النظام بهدف منع استنزاف الموارد الحاسوبية والحفاظ على استقرار الخدمة.

- طريقة التطبيق:

تم استخدام Semaphore من مكتبة Threading بقيمة ابتدائية تساوي 10. عند وصول عدد الطلبات المتزامنة إلى هذا الحد، يتم رفض أي طلب إضافي وإرجاع رمز الحالة HTTP 503 (Service Unavailable).

- التبرير:

يسمح هذا الحل بالتحكم بعدد الطلبات النشطة في اللحظة نفسها ومنع استهلاك الموارد بشكل مفرط. وقد تم اختيار القيمة 10 باعتبارها مناسبة لطبيعة النظام وعدد عمليات الشراء المتوقع تنفيذها بالتوازي.

```
order_semaphore = threading.Semaphore(10)
...
acquired = order_semaphore.acquire(blocking=False)
if not acquired:
    return JsonResponse({'error': 'Server busy, try again'}, status=503)
...
finally:
    order_semaphore.release()
```

• نتيجة الاختبار:

تم إرسال 20 طلباً متزامناً إلى النظام. كانت النتيجة نجاح 10 طلبات ورفض 10 طلبات أخرى بسبب وصول النظام إلى الحد الأقصى المسموح به من العمليات المتزامنة، مما يؤكد نجاح آلية إدارة الموارد.

NFR #2: SEMAPHORE RESOURCE MANAGEMENT TEST

NOTE: Expects semaphore=10 and a 0.7s sleep in the view.

X BUSY (503): Semaphore limit reached

X BUSY (503): Semaphore limit reached

X BUSY (503): Semaphore limit reached

X BUSY (503): Semaphore limit reached

X BUSY (503): Semaphore limit reached

X BUSY (503): Semaphore limit reached

X BUSY (503): Semaphore limit reached

X BUSY (503): Semaphore limit reached

X BUSY (503): Semaphore limit reached

X BUSY (503): Semaphore limit reached

✓SUCCESS (200) *10

]RESULTS]

Successes: 10 (expected: 10)

Busy (503): 10 (expected: 10)

Errors: 0 (expected: 0)

/// المعالجة غير المتزامنة

• المتطلب:

نقل المهام التي لا يحتاج المستخدم إلى انتظارها خارج المسار الرئيسي للطلب، مثل إنشاء الفواتير وإرسال الإشعارات.

• طريقة التطبيق:

بعد إتمام عملية إنشاء الطلب بنجاح، يتم إرسال المهام الثانوية إلى Celery ليتم تنفيذها في الخلفية. وبذلك يحصل المستخدم على الاستجابة مباشرة دون الحاجة إلى انتظار انتهاء هذه العمليات.

• التبرير:

يساهم هذا الأسلوب في تقليل زمن الاستجابة وتحسين تجربة المستخدم، حيث يتم تنفيذ المهام طويلة التنفيذ بشكل مستقل عن الطلب الرئيسي.

```
generate_invoice.delay(order.id)
send_notification.delay(request.user.id, f"Your order {order.id} was placed!")
```

• نتيجة الاختبار:

تم قياس زمن استجابة النظام أثناء إنشاء طلب جديد، وتبين أن المستخدم يحصل على الاستجابة مباشرة بينما تستمر عمليات إنشاء الفاتورة وإرسال الإشعارات في الخلفية من خلال Celery.

```
NFR #3: ASYNC TASK QUEUE TEST
=====
Response time: 2.81 seconds
Response status: 200
Response body: {'success': True, 'order_id': 1593, 'total_price': '30.00'}
```

IV. معالجة البيانات على دفعات

• المتطلب:

معالجة كميات كبيرة من البيانات بطريقة تقلل استهلاك الذاكرة وتحسن الأداء عند التعامل مع عدد كبير من السجلات.

• طريقة التطبيق:

تم استخدام Celery لتنفيذ مهمة إنشاء تقرير المبيعات اليومية في الخلفية. يتم تقسيم البيانات إلى دفعات (Chunks) بحيث تحتوي كل دفعة على 100 سجل، ثم تتم معالجة كل دفعة بشكل مستقل.

• التبرير:

يساعد هذا الأسلوب على تقليل استهلاك الذاكرة وتحسين أداء النظام عند معالجة أعداد كبيرة من الطلبات، كما يسمح بتنفيذ العملية دون التأثير على استجابة المستخدمين.

```
for i in range(0, total_orders, chunk_size):  
    chunk = orders[i:i+chunk_size]  
    chunk_revenue = sum(float(o['total_price']) for o in chunk)
```

• نتيجة الاختبار:

تم تشغيل مهمة تقرير المبيعات اليومية والتحقق من إدراجها في قائمة الانتظار الخاصة بـ Celery، ثم التأكد من أن معالجة البيانات تتم على شكل دفعات متتالية بدلاً من معالجة جميع السجلات دفعة واحدة.

```
=====
=====
NFR #4: BATCH PROCESSING TEST
=====
=====
Status: 200
Response: {'message': 'Batch job queued', 'task_id': 'ac1bbc00-c4e7-484f-a892-c0f0ad57805c'}
```

٧. توزيع الأحمال

• المتطلب:

توزيع الطلبات الواردة على أكثر من خادم بهدف تحسين الأداء وزيادة قدرة النظام على تحمل الأحمال المرتفعة.

• طريقة التطبيق:

تم استخدام Nginx كموازن أحمال (Load Balancer) أمام خادمين من نوع Waitress يعملان على المنافذ 8001 و 8002. يعتمد التوزيع على خوارزمية Round Robin التي تقوم بتوجيه الطلبات بالتناوب بين الخادمين.

• التبرير:

تم اختيار خوارزمية Round Robin لبساطتها وكفاءتها في الأنظمة التي تكون فيها تكلفة معالجة الطلبات متقاربة. كما أنها تحقق توزيعاً عادلاً للحمل دون الحاجة إلى الاحتفاظ بحالة المستخدم داخل الخادم.

• نتيجة الاختبار:

تم تشغيل أكثر من خادم تطبيق خلف Nginx ومراقبة توزيع الطلبات بين الخادمين، حيث أظهرت النتائج توزيعاً متوازناً للطلبات وفق آلية Round Robin.

```
Request 1: 200 served by port 8002
Request 2: 200 served by port 8001
Request 3: 200 served by port 8002
Request 4: 200 served by port 8001
Request 5: 200 served by port 8002
Request 6: 200 served by port 8001
Request 7: 200 served by port 8002
Request 8: 200 served by port 8001
Request 9: 200 served by port 8002
Request 10: 200 served by port
8001
Request 11: 200 served by port
8002
```

```
INFO:waitress:Serving on http://0.0.0.0:8002
```

```
[CACHE] cache miss_loading from db
```

```
[CACHE] cache hit_served from redis
```

```
[CACHE] cache hit_served from redis
```

```
[CACHE] cache hit_served from redis
```

```
[CACHE] cache hit_served from redis
```

```
[CACHE] cache hit_served from redis
```

```
[CACHE] cache hit_served from redis
```

```
[CACHE] cache hit_served from redis
```

VI. التخزين المؤقت الموزع

• المتطلب:

تقليل عدد عمليات الوصول إلى قاعدة البيانات وتحسين زمن الاستجابة للبيانات الأكثر طلباً.

• طريقة التطبيق:

تم استخدام Redis (Memurai) لتخزين قائمة المنتجات داخل الذاكرة لمدة 60 ثانية. عند وصول طلب جديد يتم أولاً البحث عن البيانات داخل Redis ، وفي حال عدم وجودها يتم جلبها من قاعدة البيانات ثم تخزينها مؤقتاً.

• التبرير:

تعد قائمة المنتجات من أكثر البيانات طلباً داخل النظام، كما أن معدل تغيرها منخفض نسبياً، لذلك فإن تخزينها مؤقتاً يساهم بشكل مباشر في تقليل الحمل على قاعدة البيانات وتحسين الأداء العام.

```
CACHE_KEY = 'all_products'
data = cache.get(CACHE_KEY)
if data is None:
    data = list(Product.objects.all().values(...))
    cache.set(CACHE_KEY, data, timeout=60)
    print("[CACHE] cache miss")
else:
    print("[CACHE] cache hit")
```

• نتيجة الاختبار:

تمت مراقبة حالات Cache Hit و Cache Miss أثناء تشغيل النظام، وتبين أن الطلبات المتكررة يتم تلبيتها مباشرة من Redis دون الحاجة إلى الوصول إلى PostgreSQL.

VII. التحكم بالتزامن والأقفال (Concurrency Control)

• المتطلب:

تطبيق آلية مناسبة للتحكم بالتزامن عند تحديث بيانات المخزون الحساسة، مع ضمان سلامة البيانات عند وجود عدة مستخدمين أو مسؤولين يعملون على النظام في الوقت نفسه.

• طريقة التطبيق:

تم تطبيق نوعين من آليات التحكم بالتزامن:

أولاً: القفل التشاؤمي (Pessimistic Locking)

يستخدم أثناء تنفيذ عمليات الشراء، حيث يتم قفل سجل المنتج باستخدام `select_for_update()` قبل تعديل كمية المخزون. يمنع هذا الإجراء أي عملية أخرى من تعديل السجل نفسه حتى انتهاء المعاملة الحالية.

ثانياً: القفل المتفائل (Optimistic Locking)

يستخدم أثناء عمليات إعادة التخزين (Restocking) التي ينفذها المسؤول. يعتمد هذا الأسلوب على حقل `version` الموجود ضمن المنتج، حيث يتم تنفيذ التحديث فقط إذا كانت النسخة المرسلة مطابقة للنسخة الحالية في قاعدة البيانات. أما في حال تغيرت البيانات بواسطة عملية أخرى فيتم رفض التحديث وإرجاع رمز الخطأ HTTP 409.

• التبرير:

تم اختيار القفل التشاؤمي لعمليات الشراء لأنها تتعرض لمنافسة عالية وتتطلب أعلى مستوى من الحماية. أما القفل المتفائل فتم استخدامه في عمليات إعادة التخزين الأقل تكراراً لتقليل زمن الاحتفاظ بالأقفال وتحسين الأداء.

```
updated = Product.objects.filter(
    id=product_id,
    version=expected_version
).update(
    stock=F('stock') + amount,
    version=F('version') + 1
)
if updated == 0:
    return JsonResponse({'error': 'stale version'}, status=409)
```

• نتيجة الاختبار:

تم تنفيذ اختبار باستخدام رقم إصدار صحيح ورقم إصدار قديم. نجحت العملية الأولى وتم رفض الثانية مع إرجاع رمز الخطأ HTTP 409 ، مما يؤكد نجاح آلية القفل المتفائل في اكتشاف التعارضات.

NFR #7: OPTIMISTIC LOCKING TEST

=====

Current version of product 4: 0

Restock with correct version: 200 - {'success': True, 'message': 'added10 to stock , new version is 1'}

Restock with stale version (-1): 409 - {'error': 'stale version or product was modified'}

VIII. سلامة المعاملات (Transaction Integrity / ACID)

• المتطلب:

ضمان أن العمليات المركبة المرتبطة بإنشاء الطلب تتم بالكامل أو لا تتم إطلاقاً، بحيث لا ينتج عن أي خطأ حالة غير متسقة داخل قاعدة البيانات.

• طريقة التطبيق:

تم تغليف جميع العمليات المرتبطة بإنشاء الطلب ضمن كتلة

`transaction.atomic()`

والتي تشمل تحديث كمية المخزون، وإنشاء سجل الطلب، وإضافة عناصر الطلب.

في حال حدوث أي خطأ أثناء تنفيذ أي خطوة من هذه الخطوات، يتم التراجع عن جميع التعديلات المنفذة سابقاً (Rollback)، مما يعيد قاعدة البيانات إلى حالتها السابقة قبل بدء العملية.

• التبرير:

يحقق هذا الأسلوب مبدأ الذرية (Atomicity) والاتساق (Consistency) ضمن خصائص ACID، ويمنع حدوث تحديثات جزئية قد تؤدي إلى فقدان البيانات أو عدم تطابقها.

```
with transaction.atomic():
```

```
    product =
```

```
    Product.objects.select_for_update().get(id=product_id)
```

• نتيجة الاختبار:

تم إجراء اختبارات باستخدام معرفات منتجات غير صحيحة وكميات غير متاحة، وتم التأكد من أن قاعدة البيانات لا تتعرض لأي تعديل جزئي عند فشل العملية، حيث يتم التراجع عن جميع التغييرات تلقائياً.

IX. اختبار النظام تحت الضغط (Stress Testing)

• المتطلب:

إثبات قدرة النظام على خدمة ما لا يقل عن 100 مستخدم متزامن دون حدوث انهيار أو فقدان للبيانات أو تدهور كبير في الأداء.

• طريقة التطبيق:

تم استخدام أداة Locust لمحاكاة 100 مستخدم افتراضي مع معدل إنشاء قدره 10 مستخدمين في الثانية ولمدة دقيقتين. تم تصميم الاختبار ليحاكي السلوك الواقعي للمستخدمين من خلال تنفيذ نوعين من العمليات:

- استعراض المنتجات عبر GET /api/products بنسبة أكبر.
- إنشاء الطلبات عبر POST /api/order بنسبة أقل.

تم اختيار هذا التوزيع لأن المستخدمين في الأنظمة الحقيقية يقضون وقتاً أطول في تصفح المنتجات مقارنة بتنفيذ عمليات الشراء.

• التبرير:

يوفر هذا السيناريو نموذجاً قريباً من الاستخدام الفعلي للنظام ويسمح بقياس استقرار الخدمات المختلفة مثل قاعدة البيانات، الكاش، الأقفال، وعمليات Celery تحت الضغط.

• نتيجة الاختبار:

أظهرت نتائج الاختبار قدرة النظام على الاستمرار في معالجة الطلبات دون حدوث أخطاء أو فقدان للبيانات، مع المحافظة على استقرار الخدمات المختلفة أثناء فترة التشغيل.

```
locust -f locustfile.py --headless -u 100 -r 10 --run-time 2m --  
host=http://localhost
```

Metric	GET /products/	POST /order/	Aggregated
Requests	1,247	312	1,559
Failures	0 (0.00%)	0 (0.00%)	0 (0.00%)
Avg (ms)	12	187	47
Min (ms)	4	82	4
Max (ms)	45	489	489
Median (ms)	11	165	23
95% (ms)	32	420	—
99% (ms)	44	480	—
RPS	41.23	10.32	51.55

X. القياس وتحليل الاختناقات والتحسين

• المتطلب:

قياس أداء النظام، تحديد نقطة اختناق واحدة على الأقل، ثم تطبيق تحسين عملي وإظهار أثره باستخدام نتائج رقمية قابلة للمقارنة.

• تحديد الاختناق:

أظهرت نتائج الاختبارات أن نقطة الاختناق الرئيسية كانت في نقطة النهاية الخاصة بإنشاء الطلبات:

POST /api/order

حيث ارتفع زمن الاستجابة عند الأحمال المرتفعة بسبب الاعتماد على القفل على مستوى الصف (Row-Level Locking) داخل قاعدة البيانات. عند ازدياد عدد الطلبات المتزامنة تبدأ العمليات بالانتظار حتى يتم تحرير القفل، مما يؤدي إلى زيادة زمن الاستجابة.

• التحليل:

بينت نتائج القياس أن عمليات قراءة المنتجات كانت أسرع بكثير من عمليات إنشاء الطلبات، وهو أمر متوقع نظراً لاعتماد الأخيرة على المعاملات والأقفال وتحديث البيانات.

بلغ متوسط زمن الاستجابة:

• GET /products حوالي 12ms

• POST /order حوالي 187ms

كما وصل زمن الاستجابة عند النسبة المئوية 95% لطلبات الشراء إلى حوالي 420ms.

• التحسين المطبق:

تمت دراسة استخدام الخيار:

select_for_update(nowait=True)

والذي يسمح بإرجاع خطأ فوري عند وجود قفل بدلاً من الانتظار حتى تحريره. يساهم هذا الأسلوب في تقليل زمن الانتظار ومنع تراكم الطلبات الطويلة.

كما أظهرت الاختبارات أن قيمة Semaphore المستخدمة كانت كافية لتحقيق استقرار النظام، وأن زيادة عدد الخيوط لم تؤدي إلى تحسين ملحوظ في الأداء مقارنة بالتكلفة الإضافية.

• النتائج:

أثبتت نتائج القياس أن النظام قادر على المحافظة على استقرار الأداء تحت الضغط مع تحقيق زمن استجابة مقبول لمعظم العمليات، كما ساهم استخدام Redis و Celery وآليات التحكم بالتزامن في تقليل الاختناقات وتحسين كفاءة المعالجة.

Endpoint	50%	66%	75%	80%	90%	95%	98%	99%	99.9%	100%
POST /api/order/	2300	2600	2700	2900	3100	3500	9600	13000	17000	17000
GET /api/products/	1600	1900	2000	2100	2300	2700	8700	12000	17000	17000

✓ الخلاصة

النتيجة	التقنية المستخدمة	المتطلب
نجاح منع التضارب	select_for_update	Race Condition Protection
10 طلبات فقط بالتوازي	Semaphore	Resource Management
تنفيذ المهام بالخلفية	Celery	Async Processing
معالجة 100 سجل لكل دفعة	Celery Chunks	Batch Processing
توزيع الطلبات بالتناوب	Nginx + Waitress	Load Balancing
تقليل الوصول لقاعدة البيانات	Redis	Distributed Cache
منع التعارضات	Optimistic + Pessimistic Locking	Concurrency Control
نجاح أو فشل العملية كاملة	transaction.atomic	Transaction Integrity
محاكاة 100 مستخدم	Locust	Stress Testing
تحديد وتحليل الاختناقات	Locust Metrics	Benchmarking

الخاتمة

قدم هذا المشروع نموذجاً عملياً لتطبيق مفاهيم البرمجة المتوازية ضمن نظام تجارة إلكترونية يعتمد على بنية خلفية حديثة وقابلة للتوسع. ركز العمل على تحقيق المتطلبات غير الوظيفية المتعلقة بحماية البيانات، إدارة الموارد، تحسين الأداء، وضمان استقرار النظام تحت الأحمال المرتفعة.

تم استخدام مجموعة من التقنيات المتكاملة شملت Django و PostgreSQL و Redis و Celery و Nginx و Locust، حيث ساهم كل منها في تحقيق جانب محدد من متطلبات النظام. كما تم تطبيق آليات مختلفة للتحكم بالتزامن مثل القفل التبادلي والقفل المتبادل، بالإضافة إلى استخدام المعاملات الذرية (Transactions) لضمان سلامة البيانات أثناء تنفيذ العمليات الحساسة.

أظهرت نتائج الاختبارات نجاح النظام في منع حالات التضارب عند الوصول المتزامن إلى البيانات، وإدارة الموارد الحاسوبية باستخدام Semaphore، وتنفيذ المهام غير المتزامنة بواسطة Celery، ومعالجة البيانات على شكل دفعات، والاستفادة من التخزين المؤقت لتحسين الأداء. كما أثبتت اختبارات الضغط قدرة النظام على التعامل مع عدد كبير من المستخدمين المتزامنين مع المحافظة على استقرار الخدمات الأساسية.

توضح نتائج المشروع أهمية الجمع بين تقنيات التحكم بالتزامن وإدارة الموارد والتخزين المؤقت والمعالجة غير المتزامنة لبناء أنظمة برمجية قادرة على العمل بكفاءة وموثوقية في البيئات الحقيقية ذات الأحمال المرتفعة.

وبذلك يكون المشروع قد حقق الأهداف المطلوبة وقدم تطبيقاً عملياً لمعظم المفاهيم النظرية التي تمت دراستها ضمن مقرر البرمجة المتوازية.

The End